

KRJS Production AI Product Capstone

Karunada Raitha Jana Abiruddhi Sanga (KRJS) Website

Name: Pradeep S

Project status: Existing React/Vite website being finalized for production.

1. Project Brief

The KRJS website is a professional digital platform for Karunada Raitha Jana Abiruddhi Sanga, a farmer organization focused on empowering farmers and supporting agricultural and rural development in Karnataka. It is intended for farmers, community members, supporters, and other stakeholders who need clear information about the organization, its mission, leadership, activities, achievements, and events. The capstone extends the existing frontend with one meaningful AI capability, resilient states, testing, accessibility and performance checks, and a verified production deployment.

2. Existing Application

The current project uses React and Vite with Tailwind CSS, Framer Motion, and Lucide React. It is organized into reusable sections/components such as Navbar, Hero, About, Stats, Committee, Works, and Events, with data files for committee, works, and events. Planned backend/database/admin work is not treated as completed unless actually implemented.

3. Capstone Scope

Area	Required outcome	Status
Frontend	Accessible, responsive, reusable React UI	Existing; audit/polish required
AI	Meaningful AI capability solving a farmer/KRJS information problem	Not yet implemented
Resilience	Loading, error, empty, invalid-input and fallback states	Required
Testing	At least one real component test or critical E2E flow	Not yet configured
Accessibility	WCAG 2.1 AA target; axe/WAVE evidence	Audit required
Performance	Lighthouse ≥85 target; aim 90+	Baseline must be rerun
Deployment	GitHub + production Vercel/Netlify URL	Not yet verified
README	Setup, architecture, AI, limitations and deployment	Must be completed

4. AI Integration Plan

AI Farmer Resource Assistant: users ask questions about KRJS programs, farmer resources, activities, or available website information. A protected server-side AI endpoint sends the question with approved KRJS/farmer context and returns a structured answer. The feature should not invent unavailable facts and should clearly say when relevant information is unavailable.

Input	Farmer/KRJS question.
Prompt	Answer from approved KRJS/farmer information; concise; no invented facts.
Output	Structured answer with relevant resource/category and fallback when unsupported.
Security	API key server-side only; no secrets in browser/repository.
Failure	Loading, timeout/API error, invalid response, retry/fallback states.

5. Error, Empty and Edge States

Handle empty questions, long input, unsupported questions, failed AI requests, timeouts, malformed model output, repeated submissions, network failure, and missing content. Each state should give a clear next action such as retry, edit the question, or navigate to a relevant KRJS section.

6. Testing Plan

Use Vitest/Jest or another React-compatible test tool. At minimum, create one real component test or critical AI user-flow test and capture its output/coverage. Where practical, also test navigation, AI loading/error behavior, and the main user interaction.

7. Performance & Accessibility

Run Lighthouse after the production UI is complete. Target ≥85 and aim for 90+. Run axe DevTools or WAVE. Check keyboard access, visible focus, semantic HTML, labels, button names, contrast, responsive layout, image alternatives, and accessible error messages. Record at least one concrete audit-driven improvement.

Metric	Current recorded value	Required action
Performance	10	Rerun after fixes; target ≥85
Accessibility	5	Rerun and resolve violations
Best Practices	8	Rerun and improve
SEO	2	Rerun and improve metadata/structure

8. Related Resources — Applied to This Capstone

The ten supplied resources are implementation guidance. They are not claimed as completed work until the corresponding feature, audit, test, or deployment evidence actually exists.

#	Resource	Application
1	Frontend Fundamentals (The AI-Proof Foundation)	Component structure, reusable UI, semantic HTML and maintainable frontend patterns.
2	The Core Web Pillars (Perf, SEO, & Accessibility)	Lighthouse, SEO metadata, performance optimization and accessibility verification.
3	The AI-assisted workflow drill	AI-assisted planning, implementation, debugging and review; document actual use.
4	Tool results and structured output in the UI	Structured AI results rather than raw model text; predictable UI states.
5	Error states, empty states, edge cases	Loading, empty, invalid-input, timeout, API-error and fallback states.
6	React Frontend Development with AI: From Prompt to Working Feature	Guide the React AI feature from requirement to working UI.
7	Capstone skeleton, deployed	Deployment-ready structure, environment-variable discipline and route verification.
8	Streaming AI chat interface	Use only if streaming improves the selected interaction; otherwise keep a simpler flow.

#	Resource	Application
9	React app development with AI	AI-assisted React development while keeping code understandable and documented.
10	Accessible component fundamentals	Keyboard access, focus, labels, semantics, contrast and accessible behavior.

9. Deployment & Operations

Target platform: Vercel or another supported production host. The final submission must contain an actual production URL, not localhost. The repository must contain complete source and a README that lets a new developer install and run the project. API keys must be deployment environment variables and excluded from source control.

Checklist	Status before final verification
■ Production build succeeds	To be verified
■ GitHub repository contains complete source	To be verified
■ README setup works on a clean environment	To be verified
■ AI/API key protected	To be verified
■ AI loading state	To be verified
■ AI error/timeout state	To be verified
■ Empty/invalid input handled	To be verified
■ Mobile layout tested	To be verified
■ Keyboard accessibility tested	To be verified
■ axe/WAVE audit completed	To be verified
■ Lighthouse audit completed	To be verified
■ At least one real test passes	To be verified
■ Production URL verified	To be verified
■ Rollback procedure documented	To be verified

Rollback plan: revert the problematic commit/deployment and redeploy the last known-good version, recording the version used for rollback.

10. Repository & Production URLs

Repository URL	To be added after GitHub repository is created
Production URL	To be added after Vercel/hosting deployment is verified

11. README Requirements

Include project overview, audience, local setup, install/run command, architecture, AI integration and prompt purpose, environment variables, error handling, testing command/evidence, accessibility/performance results, deployment instructions, limitations, future improvements, and production URL.

12. Known Limitations & Future Improvements

Current documentation records that the website is under development, some sections such as Gallery and Contact are not fully implemented, backend/database/admin work is planned, automated testing and formal accessibility evidence are missing, and AI is not yet integrated. These should remain limitations unless actually completed. Future improvements include Kannada support, secure content management, backend/admin functionality, and further production hardening.

13. Reflection

What was hardest? Refining the frontend while keeping React components consistent required attention to section IDs, navigation links, spacing, and component styling. The capstone adds the challenge of making the application resilient and production-verifiable rather than stopping at visual completion.

What would I do differently next time? Establish shared layout, navigation, spacing, typography, button, card, testing, accessibility, and deployment patterns earlier so production checks are part of development.

One thing I learned that surprised me: A small implementation detail such as a section ID can have a major effect on user experience. The same applies to production details such as accessibility, error states, environment variables, and deployment.

14. Final Submission Checklist

- Project brief completed
- KRJS application works end-to-end
- Meaningful AI feature implemented and tested
- AI output structured with safe fallback
- Error/empty/loading/edge states implemented
- At least one real test passes
- Lighthouse ≥85 verified
- Accessibility audit completed
- One audit-driven improvement documented
- README completed
- Deployment checklist signed off
- GitHub repository verified
- Production deployment verified
- Rollback approach documented
- Reflection completed.